Pioneer.jl Flanking-Window Features

Feature reference

2026-06-24

1 Motivation: wide-isolation-window DIA

In data-independent acquisition with *wide* quadrupole isolation windows, a single MS2 window co-isolates many precursors. A confident identification therefore needs evidence that the precursor's signal is real and not bleed-through from a co-isolated neighbor or chromatographic interference.

The **flanking-window features** provide this evidence by contrasting two regions of the *same* isolation window:

- the **core**: the contiguous run of MS2 cycles, centered on the precursor's apex scan, where the precursor passes the search threshold;
- the **flank**: scans of the same isolation window in the cycles immediately *flanking* the core (just outside the core's scan range).

A genuine precursor has its MS1 and fragment signal concentrated in the core, with fragments that co-elute; interference tends to smear signal into the flank or to show fragments that do not track the precursor. The features quantify the core-vs-flank contrast and the flanking-scan co-elution structure.

2 Where they are computed (two pipeline stages)

1. **MainSearch** (`SearchMethods/MainSearch/scoring.jl`) — after best-per-precursor selection, `_mainsearch_flanking_core_bounds!` (`:811`) determines the *core* window and the per-precursor loop (`:1045--1093`) writes the intermediate "core" columns into the best-PSM table, which are persisted in the fold Arrow files: `flanking_core_scan_min`, `flanking_core_scan_max`, `n_contiguous_scans`, `flanking_core_ms1_m0_signal`, `flanking_core_frag_signal`.
2. **PrecursorScoringSearch** (`PrecursorScoringSearch.jl:199` calls `add_wide_window_features_to_fold_f` which calls `add_wide_window_features_to_fold_file!` once per fold file per MS run, in `wide_window_features.jl`). It reads the core columns, gathers the flanking scans, extracts MS1 + fragment intensities from the raw spectra, computes the 11 flanking feature values in `_wide_flank_feature_values` (`:58`), and writes them back to the fold Arrow file. The two `*_signal` core columns are intermediate and dropped afterward (`:893`).

The final feature set (the canonical list `FLANKING_WINDOW_FEATURES`, `model_config.jl:49`) is then consumed by the second-pass LightGBM in ScoringSearch.

3 Defining the core (MainSearch)

`_mainsearch_flanking_core_bounds!` takes the rows of one precursor group (all scans where the precursor was scored), sorts them by `(cycle_idx, scan_idx)`, locates the *best* (selected/apex) scan, and walks outward while each neighbor (a) passes the mask and (b) is in the immediately adjacent cycle (± 1):

```
# walk left while passing & cycle is exactly one less
while lo>1 && pass_mask[prev] && cycle[prev]==cycle[cur]-1; lo-=1; end
# walk right while passing & cycle is exactly one greater
while hi<len && pass_mask[next] && cycle[next]==cycle[cur]+1; hi+=1; end
```

It returns `scan_min`/`scan_max` (the core scan range), `n_scans` (=`n_contiguous_scans`), and the boundary rows. The core MS1 and fragment *signals* are then summed over rows whose `scan_idx` lies in [scan_min, scan_max] (`scoring.jl:1086--1093`):

$$\text{core\_ms1\_m0\_signal} = \sum_{\text{core scans}} \text{ms1\_m0}, \qquad \text{core\_frag\_signal} = \sum_{\text{core scans}} \left(\sum_{r=1}^{8} \text{frag}_r^{\text{smoothed}} \right).$$

The 8 core fragment intensities are the smoothed fragment-chromatogram apex values `frag1..8_smoothed_intensi` (`SMOOTHED_FRAGMENT_INTENSITY_COLUMNS`).

4 Gathering the flank and extracting signal (PrecursorScoringSearch)

- `_wide_window_index_spectra` (`:286`) indexes every MS2 scan by its isolation window key `(center_mz, isolation_width)`, giving the set of scans belonging to the same wide window across all cycles.
- `_wide_group_flank_window_scans_from_core!` (`:420`) collects, for the precursor group, the same-window scans in cycles flanking the core (outside [scan_min, scan_max], within `WIDE_WINDOW_CYCLE_MARG` 3 cycles). If there are no flank scans, all features take their zero/default values (`_wide_window_zero_feature_v`
- `_wide_top_fragment_idxs` + `_wide_group_fragment_windows` pick the precursor's top-8 library fragments and their m/z tolerance windows.
- Scan-centric extraction (`_wide_fill_scan_centric_ms1_m0!`, `_wide_fill_scan_centric_fragments!`, batched at `FLANKING_SCAN_CENTRIC_BATCH_SIZE`= 50000 groups via `_wide_flush_scan_centric_group_batch` reads each flank scan once and fills `flank_ms1_m0` (length = #flank scans) and `flank_fragments` (#flank scans $\times$ 8).

5 The feature computations (`_wide_flank_feature_values`, line 58)

Let the inputs be: core scalars c_{m0} (`core_ms1_m0_signal`) and c_f (`core_frag_signal`); the 8 core apex fragment intensities a_r (`core_fragments`); the flank MS1 vector $\mathbf{m}$ (`flank_ms1_m0`, one value per flank scan); and the flank fragment matrix F (`flank_fragments`, scan $\times$ fragment). All intensities are floored at 0. Define $s_j = \sum_r F_{j,r}$ (summed fragment signal in flank scan j), and let $\rho(\cdot,\cdot)$ be Pearson correlation (`_wide_window_pcor`, returns 0 for < 2 points or zero variance).

Feature	Type	Definition & intuition
flanking_ms1_m0_candidate_fraction	Float32	$\frac{c_{m0}}{c_{m0} + \sum_j \mathbf{m}_j}$ — fraction of the precursor's MS1 monoisotopic (m0) signal that sits in the core vs. spread into flank scans. $\to 1$ for a clean precursor.
flanking_frag_candidate_fraction	Float32	$\frac{c_f}{c_f + \sum_j s_j}$ — same core-vs-flank fraction for summed fragment signal.
flanking_ms1_frag_sum_corr	Float32	$\rho(\mathbf{m}, \mathbf{s})$ over flank scans — do the MS1 precursor and the fragment sum co-vary across the flank? (co-elution sanity check off-apex).
flanking_frag_corr_mean	Float32	mean of all pairwise $\rho(F_{:,a}, F_{:,b})$ over flank scans, among fragments that have signal — average fragment-fragment co-elution in the flank.
flanking_n_correlated_fragments	UInt8	count of fragments whose leave-one-out correlation $\rho\big(F_{:,r},\, \mathbf{s} - F_{:,r}\big) > 0.7$ (WIDE_WINDOW_CORR_THRESHOLD) — how many fragments track the rest.
flanking_n_correlated_fragments_bitvec_pattern_rank	UInt16	bitvec_pattern_rank of the 8-bit mask of *which* fragments exceeded 0.7 — a learned categorical encoding of the correlated-fragment *pattern* (not just the count).
flanking_frag_corr_strength	Float32	$\sum_r \max(\min(\rho_r, 1), 0)$ where ρ_r is the leave-one-out correlation — continuous total correlation strength across the 8 fragments.
flanking_frag_corr_effective_n	Float32	$\frac{(\sum_r \rho_r^+)^2}{\sum_r (\rho_r^+)^2}$ (participation ratio of the positive leave-one-out corrs) — effective number of co-eluting fragments.
flanking_frag_corr_best_m0	Float32	$\rho(F_{:,b,\,\mathbf{m})}$ where b is the fragment with the highest mean pairwise correlation (the "anchor") — does the best fragment track the MS1 precursor across the flank?
flanking_signal_support	Float32	$\frac{\#\{j : \mathbf{m}_j > 0 \text{ or } s_j > 0\}}{\#\text{flank scans}}$ — fraction of flank scans carrying any signal.
frag_apex_gt2x_flank_bitvec_rank	UInt16	_bitvec_pattern_rank of the 8-bit mask of fragments whose *core apex* $a_r \geq 2\times$ their flank average $\bar{F}_{:,r}$ — which fragments genuinely peak at the apex rather than being flat/interference.

Feature	Type	Definition & intuition
n_contiguous_scans	UInt16	(from the core bounds, not _wide_flank_feature_values) the number of contiguous cycles in the core run — how many cycles the precursor persisted.

The 11 values returned by _wide_flank_feature_values plus n_contiguous_scans make up the 12-entry FLANKING_WINDOW_FEATURES list.

6 Helper functions

Function (wide_window_features.jl / scoring.jl)	Role
_mainsearch_flanking_core_bounds! (scoring.jl:811)	Find the contiguous-cycle core window around the apex; return scan_min/max, n_scans, boundary rows.
_wide_window_pcor (:5)	Single-pass Float32 Pearson correlation; 0 if < 2 points or zero variance.
_wide_candidate_signal_fraction (:32)	core/(core + flank), guarding divide-by-zero.
_wide_window_zero_feature_values (:37)	Default feature tuple when there are no flank scans (fractions still computed from the core signal).
_wide_flank_feature_values (:58)	Compute the 11 flank features from core + flank arrays.
_wide_window_index_spectra (:286)	Index scans by isolation window (center_mz, isolation_width).
_wide_group_flank_window_scans_from_cohort! (:420)	Collect same-window scans in cycles flanking the core (margin = 3 cycles).
_wide_top_fragment_idxs / _wide_group_fragment_windows (:453/:511)	Select the top-8 library fragments and their m/z extraction windows.
_wide_fill_scan_centric_ms1_m0! / _wide_fill_scan_centric_fragments! (:578/:601)	Read raw-spectra MS1 m0 and fragment intensities at each flank scan (scan-centric, batched).
_wide_scatter_features! (:706)	Write the computed feature tuple to every row of the precursor group.

Function (`wide_window_features.jl` / `scoring.jl`)	**Role**
`_bitvec_pattern_rank`	Map an 8-bit fragment pattern mask to a learned rank (file-specific table, `getBitVecExcessRanks`).

7 Constants

`WIDE_WINDOW_CYCLE_MARGIN`= 3 (flank reaches up to 3 cycles either side); `WIDE_WINDOW_CORR_THRESHOLD`= 0.7 (correlated-fragment cutoff); `FLANKING_SCAN_CENTRIC_BATCH_SIZE`= 50000 (groups per scan-extraction batch).

8 Consumption

The features are appended to the per-fold second-pass PSM Arrow files and become inputs to the ScoringSearch second-pass LightGBM (via `FLANKING_WINDOW_FEATURES`). They are most informative on wide-window instruments (e.g. Sciex SWATH), where co-isolation is heaviest.